# Modular ML-QEM Approach

A Highly Modular Pipeline for Quantum Error Mitigation with Machine Learning

---

Aniket Khan(ME21B021

Shubham Aggarwal(ME21B192)

March 23, 2025

# Overview

- **Modularity is Key:** The project is designed from scratch in a very modular way, making troubleshooting and monitoring of each step easy.
- Two main components:
  - **Data_gen Module**
    - `converter.py`
    - `extractor.py`
    - `generator.py`
    - `simulator.py`
    - `zne.py`
  - **QEM Module**
    - `base_models.py`
    - `trainer.py`
    - `preprocessor.py`
    - `evaluator.py`
- Additional scripts are provided to tune and log hyper-parameters of the ML models.

# DATA_GEN Module (Part 1)

- **converter.py:** Takes a Qiskit circuit and converts it into one of three representations:
  - **Frequency-based:** Counts gate frequencies and bins gate-parameters with customizable options (per gate, per attribute, or global).
  - **Graph-based:** Prepares data for a graph neural network.
  - **CNN-based:** Structures data as $in\_channel \times n\_qubits \times depth$, with special handling for multi-qubit gates by assigning master/slave tags.
- **extractor.py:** Extracts a machine learning applicable dataset from the raw data. This raw dataset is dictionary-based and serves as a manual sanity check (e.g., verifying proper gate implementation, one-hot encoding, etc).
- **generator.py:** Orchestrates the synthesis of new circuit data and manages simulation workflows.

# DATA_GEN Module (Part 2)

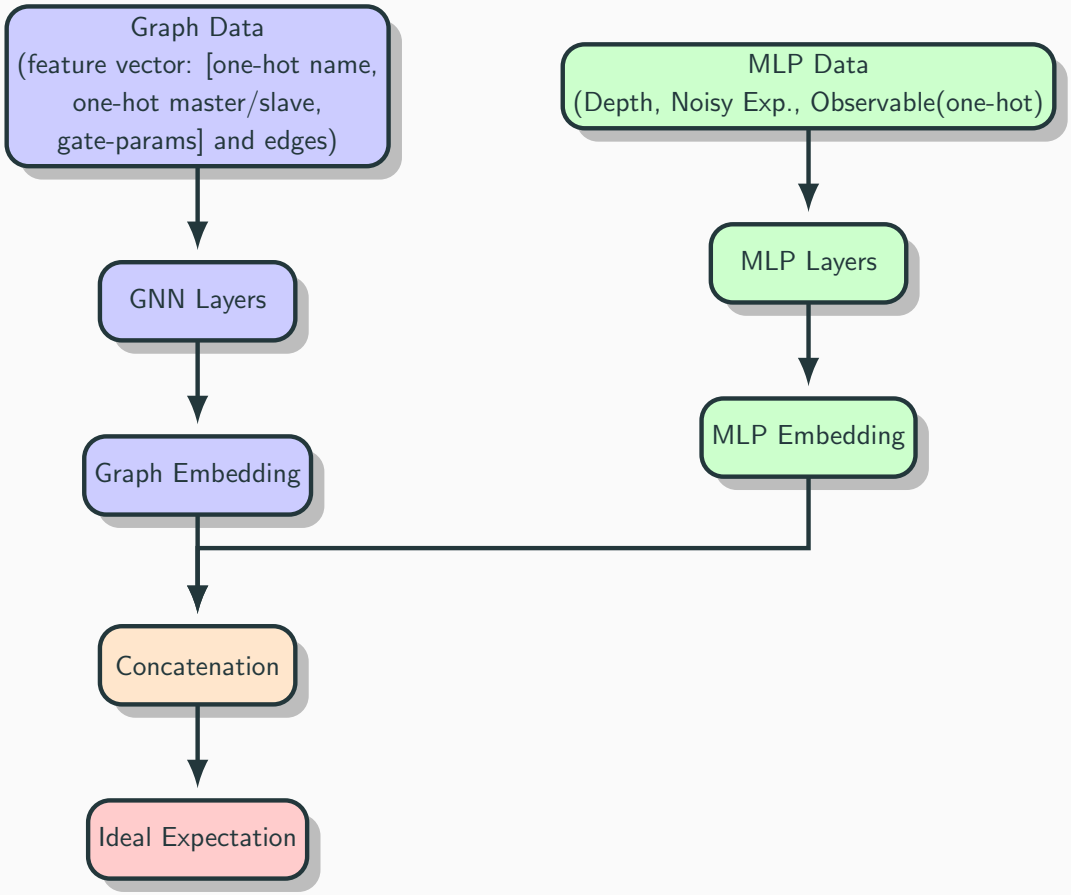
- **simulator.py:** Executes circuit simulations using AerSimulator with customizable parameters (shots, depth, number of circuits). For ideal simulations; state-vector method ensures consistency. It also supports both random and fixed-sequence Pauli observables based on experimental requirements. Additionally, the module offers the option to use either IBM calibration data as the noise model or a AER Noise model. The class accepts a transpilation argument, ensuring that the generated circuits are compatible with actual hardware (default backend: `GenericBackendV2`).
- **zne.py:** Applies Zero Noise Extrapolation and supports various extrapolation methods, including linear and Richardson, with adjustable noise-scaling parameters.

# QEM Module

- **base_models.py:** Consists of basic classes that can be imported by `trainer.py` to build complex models.
- **trainer.py:** End-to-end implementation of ML algorithms (currently, only a hybrid GNN is implemented).
- **preprocessor.py:** Separates the data generation from the ML part. It reads from a CSV/JSON/PKL file generated by `DATA_GEN`.
- **evaluator.py:** Evaluates model performance on QEM tasks (e.g., box plots, mean/std plots).

**Note:** The entire project is built on qiskit's library and mitiq library for ZNE. The implementation for hyperparameter tuning of ML models is done using `optuna` library and logging data by `wandb` library.

# Hybrid GNN Architecture

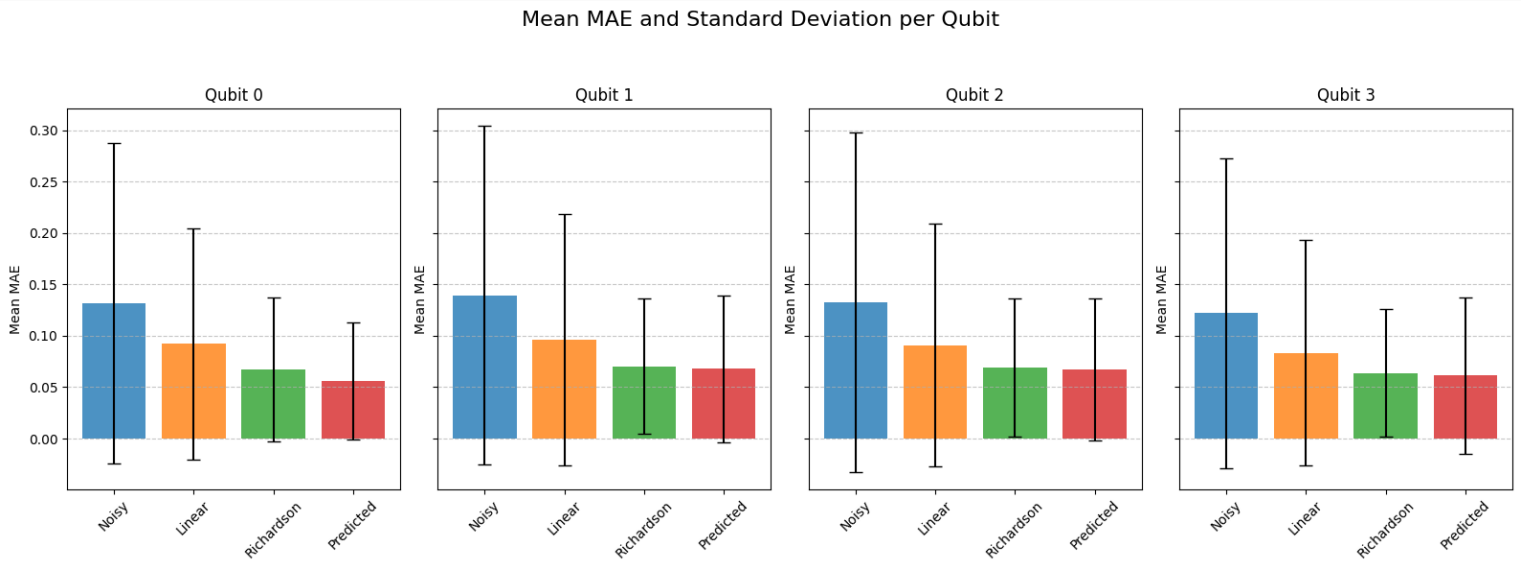



**Hybrid GNN Architecture:** Graph-based circuit data and MLP features are merged to predict the ideal expectation.

# 4 Qubits, 10 Depth, 1000 Data-Samples, Custom Noise Model

- **Error Metrics (Combined):**
  - **Noisy:** RMSE = 0.2065, MAE = 0.1316, MSE = 0.0427
  - **Linear:** RMSE = 0.1470, MAE = 0.0908, MSE = 0.0216
  - **Richardson:** RMSE = 0.0948, MAE = 0.0677, MSE = 0.0090
  - **Predicted:** RMSE = 0.0936, MAE = 0.0632, MSE = 0.0088
- **Observations:**
  - The modular design facilitates quick identification and tuning of performance bottlenecks.
  - Noise mitigation effectiveness shows progressive improvement from Noisy to Linear, then Richardson, and finally Predicted methods.
  - Training the ML model and predicting the ideal expectation requires orders of magnitude less time. For example, generating 4-qubit data with simulation and ZNE takes about 2 hours, whereas the ML model completes prediction in roughly 5 minutes(provided dataset is already generated)

# Visual Performance Analysis



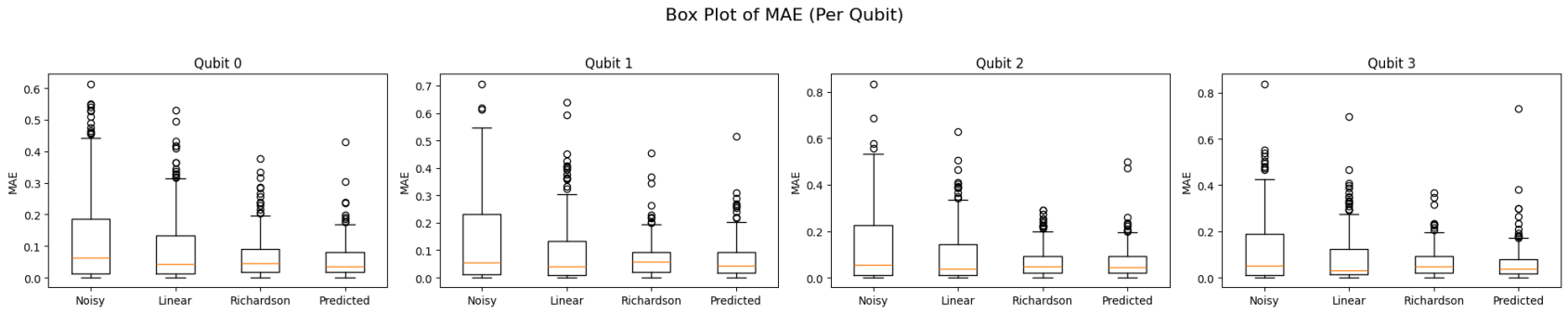




**Figure 1:** Mean and Standard Deviation across each qubit

**Figure 2:** Box plot of errors across each qubit for the four methods

# Future Work & Challenges

- Develop additional CNN and traditional ML-based algorithms.
- Explore novel encoding methods to support a wider range of models.

**Current Challenges:**

- Data generation is taking an excessively long time; over 500 minutes have elapsed without completing the generation of 10 circuits for 20 qubits.
- The current deployment environment has version issues: although Python 3.11.11 is installed on the Aqua cluster, every session reverts to Python 2.6.6, which does not support most of the required libraries.